Date No

Week 04

01] Decision tree model

*) A decision tree is a supervised learning algorithm used for classification and regression tasks.

*) It works splitting data into smaller subsets based on feature values, forming a tree like structure :

a) Root node :- Represents the entire dataset

b) Internal nodes :- Represent decisions (test on feature)

c) Branches :- Represent outcomes of decisions

d) Leaf nodes :- Represent final predictions

Goal => Create a model that predicts the target variables by learning simple decision rules infered from the data features

ex :- Email spam detector
Root : Does the email contain "free" ?
Branch => yes : Spam likely
no : try another feature (Has an attachment?)
(Check)

02] Decision tree Learning process

(divide & Conquer approach)

1] Select the root node with the full training data.

2] Select the best feature to split the data based on purity measure

3] Partition the dataset into subsets according to the feature's possible values

4] Repeat steps 2-3 for each subset unit (When do you stop splitting):

*] When a node is 100% one class
*] When splitting a node will result in the tree exceeding a maximum depth
*] When improvements in purity score are below a threshold
*] When a number of examples in a node is below a threshold

5] Assign class labels (predictions) to the leaf nodes

Date No

03] Measuring Purity

*) A pure node contains samples of only one clas while an impure node contains a mix

Entropy as a measure of impurity

*) Measures uncertanity, impurity of a node

P_1 = fraction of examples that are cats (cat detector)

$$P_0 = 1 - P_1$$

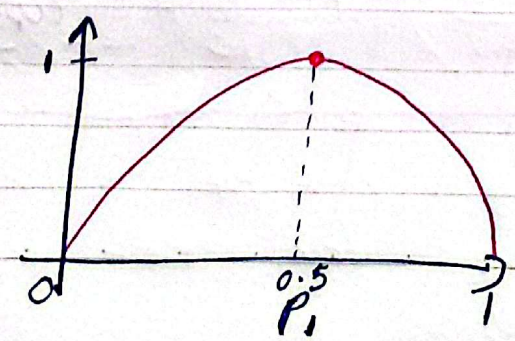
$$H(P_1) = -P_1 \log_2(P_1) - P_0(\log_2(P_0))$$

$$H(P_1) = -P_2 \log_2(P_2) - (1-P_1) \log_2(1-P_1)$$

(Entropy)

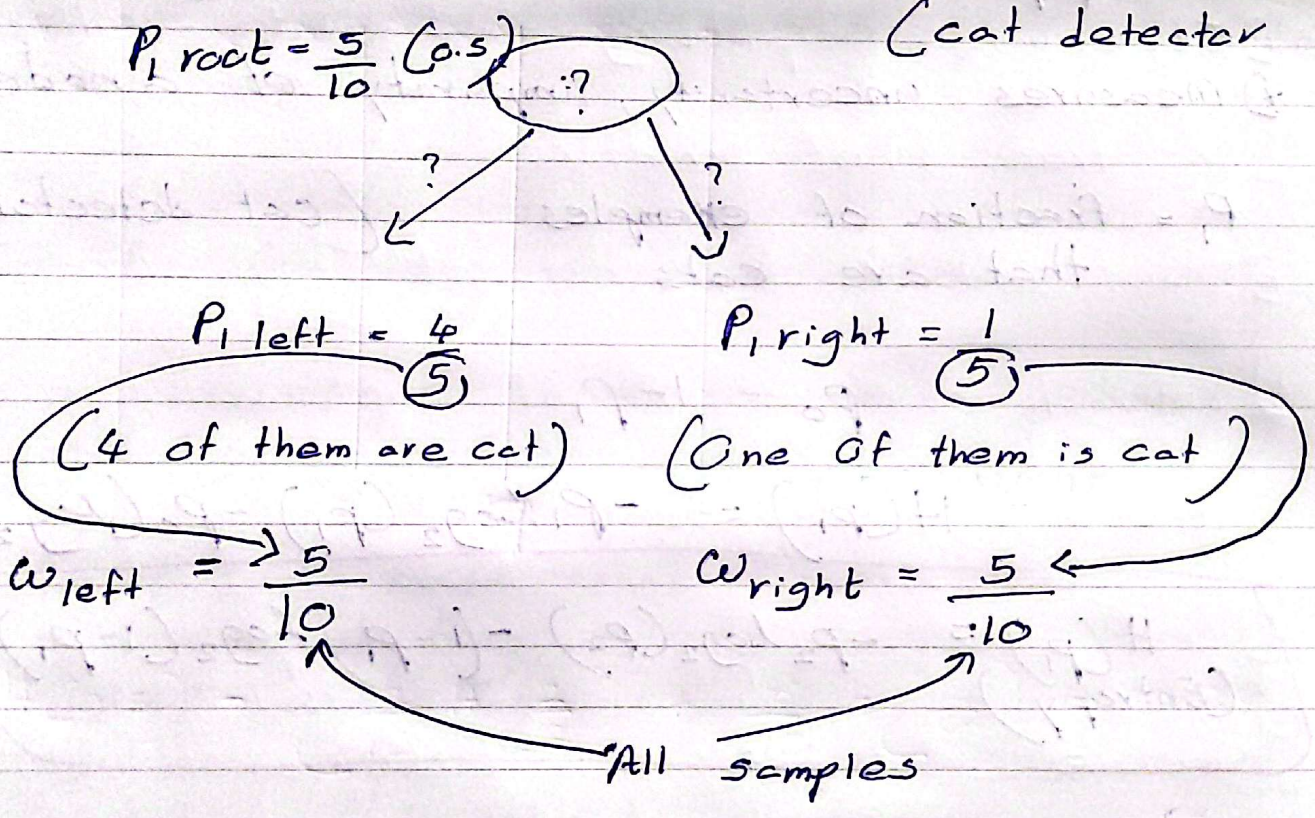
Entropy = 0 → Completly pure (only one class)

Entropy = 1 → Completly impure (50-50 split for two classes)



4) Chose the best split

*) To chose what feature to use the split we calculate information gain and chose the feature split with highest information gain



$$\text{Information gain} = H(p_1^{root}) - \left(w_{left} \cdot H(p_1^{left}) + w_{right} \cdot H(p_1^{right})\right)$$

H= entropy

Date No

Workflow

1] Start with all examples at the root node.

2] Calculate the information gain for all possible features, and pick one with the highest information gain.

3] Split dataset according to selected feature, and create left and right branches of the tree

4] keep repeating until one of the stopping criteria met.

5] One hot encoding

*] If a categorical feature can take on k values, create k binary features (0 or 1 valued)

Color => Red, Green, blue

	R	G	B
Color(R)	1	0	0
Color(G)	0	1	0
Color(B)	0	0	1

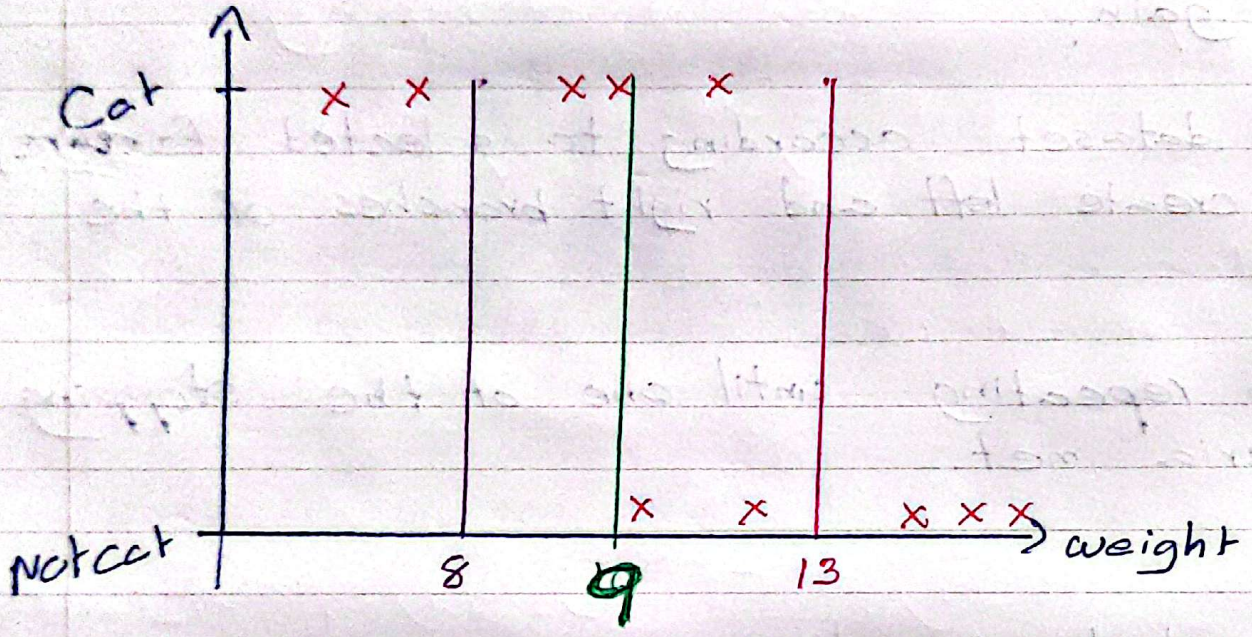
Instead of :

colours
Red
Green
Blue

=> we can One hot encode by Creating new Columns with those categories/ran

6) Continuos Valued Variables

If we found Continuos values like weight of cat (8, 8.5, 9, 18, 20) we cannot ~~Appr~~ split like normal categorical features



$$H(0.5) - \left(\frac{2}{10} H\left(\frac{2}{2}\right) + \frac{8}{10} H\left(\frac{3}{8}\right)\right) = 0.24$$

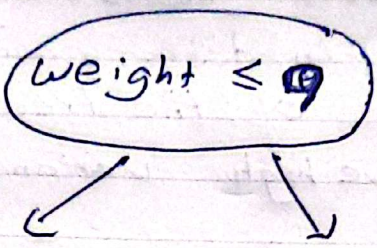
$$H(0.5) - \left(\frac{4}{10} H\left(\frac{4}{4}\right) + \frac{6}{10} H\left(\frac{1}{6}\right)\right) = 0.61$$

$$H(0.5) - \left(\frac{7}{10} H\left(\frac{5}{7}\right) + \frac{3}{10} H\left(\frac{0}{3}\right)\right) = 0.40$$

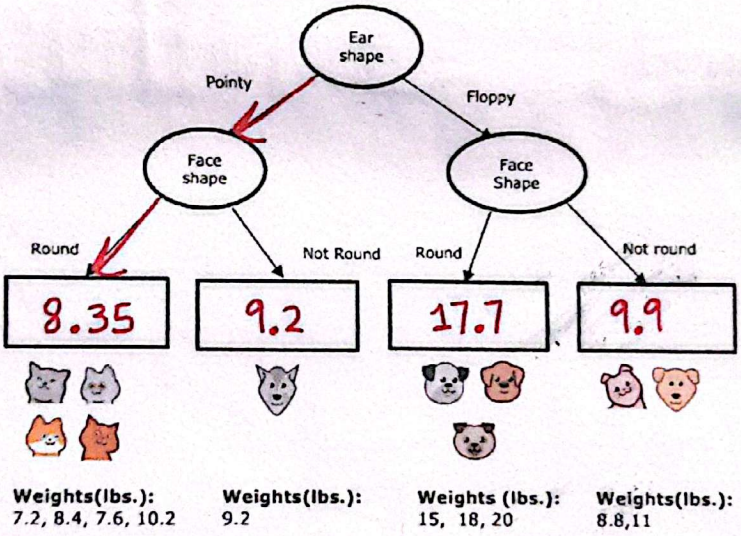
* If there are (n) number of continuos value we try (n-1) times for each value and Calculate information gain

x: 10 weights
we calculate IG for 9 weights

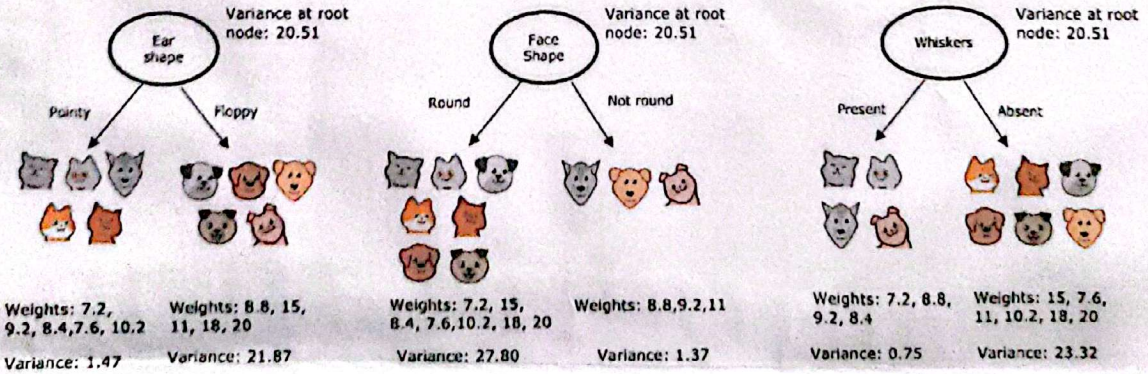
) Then we chose the weight (continuos value) with highest IG and split



) Regression Trees:



Choosing a split



$w^{left} = 5/10$ $w^{right} = 5/10$

$20.51 - \left(\frac{5}{10} * 1.47 + \frac{5}{10} * 21.87\right)$

$= 8.84$ ←

$w^{left} = 7/10$ $w^{right} = 3/10$

$20.51 - \left(\frac{7}{10} * 27.80 + \frac{3}{10} * 1.37\right)$

$= 0.64$

$w^{left} = 4/10$ $w^{right} = 6/10$

$20.51 - \left(\frac{4}{10} * 0.75 + \frac{6}{10} * 23.32\right)$

$= 6.22$

Date No

8] Multiple Decision Trees (Ensembles)

*) A single decision tree is simple and interpretable but its often not robust
→ It can overfit (memorizing data)
→ Small changes in data can cause big changes in the structure of the tree
→ It may have high variance

*) To overcome this, we use ensemble methods, which combine many decision trees to create stronger, more accurate and more stable model.

Ensemble Learning = Combining predictions from multiple models (weak learners) to form a strong learner.

Two main types

1) Bagging - used in Random Forests
2) Boosting - used in XGBoost, AdaBoost - etc.

Date No

9] Bagging and Sampling with replacement

*] Instead of training one tree on all the data, we train many trees, each on a random sample of the data.

*] This random sampling is done with replacement, known as "bootstrap sampling".

Workflow

1] From the original data set of size N, randomly draw N samples with replacement
(so some samples repeat, others are left out)
- Each tree sees slightly different dataset

2] Train an independent decision tree on each sample.

3] For prediction
- Classification: use majority voting across trees
- Regression: use average of all tree predictions

Goal: Reduce variance (overfitting) by averaging many slightly different trees

10] Random Forest algorithm

Random Forest = Bagging + Random Feature Selection

1) For each tree :
*) Take a bootstrap sample of the data
*) At each node split, instead of testing all features
→ Randomly select a subset of features ($\sqrt{n}$, $n/3$)
*) Find the best split only among those selected features

2) Build many such trees

3) Predictions :
*) Classification → majority vote
*) Regression → average output

11] Boosting (Sequential Tree ensembling)

*) while bagging building trees indepnendtly, boosting builds them sequentialy

*) Each new tree is trained to correct the mistakes made by the previous one

Date No

12] XGBoost (Extreme Gradient)

1] Start with initial prediction
2] Compute residuals (true value - current prediction)
3] Fit a new decision tree to predict those residuals
4] Update predictions by adding the new tree's output (scaled by a learning rate)
5] Repeat for many rounds until performance stops improving

*) Use gradient based, optimization and regularization to prevent overfiting
*) Reduces both bias and variance

13] When to use Decision Trees vs Neural Networks

*) works well on structured data
*) Not recommend for unstructured data (Image, audio, text)
*) Fast
*) Small DTs are human interpretable

} Decision Trees & Tree Ensembles

*) Works well on all type of data
*) May be slower than DTs
*) Works with transfer learning
*) When building a system of multiple models working together, it might be easier to String together multiple NN

Neural Networks